

Pierre-Louis Lieske

Rapport de Projet : Déploiement Kubernetes

Ce mini-rapport a pour objectif de documenter la réalisation du projet de déploiement d'une application conteneurisée (via une image Docker) au sein d'un cluster Kubernetes. Il regroupe les captures d'écran attestant du bon fonctionnement de l'application sur Kubernetes, ainsi que l'adresse du dépôt GitHub contenant le projet.

1. Introduction

Ce projet a consisté à utiliser une image Docker existante pour la déployer sur un environnement géré par Kubernetes. Les étapes clés incluent la création du déploiement et l'exposition du service pour rendre l'application accessible.

2. Déploiement Kubernetes et Exposition du Service

Pour ce projet, l'image Docker suivante a été utilisée : [5pierre/firstnameservice](#)

Vérification des noeuds et déploiement à partir de l'image Docker :

```
kubectl get nodes
```

```
kubectl create deployment myservice --image=5pierre/firstnameservice
```

```
PS C:\Windows\system32> kubectl get nodes
NAME        STATUS    ROLES    AGE   VERSION
minikube    Ready     control-plane  8h   v1.34.0
PS C:\Windows\system32> kubectl create deployment myservice --image=5pierre/firstnameservice
deployment.apps/myservice created
PS C:\Windows\system32>
```

Vérification de l'état du pod et récupération des détail et log du pod:

`kubectl get pods`

`kubectl describe pods`

```
PS C:\Windows\system32> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
myservice-5d9447b574-4xcjv         1/1     Running   0           9m42s
PS C:\Windows\system32> kubectl describe pods
Name:                                myservice-5d9447b574-4xcjv
Namespace:                          default
Priority:                            0
Service Account:                    default
Node:                                minikube/192.168.49.2
Start Time:                         Tue, 06 Jan 2026 20:04:09 +0100
Labels:                             app=myservice
                                     pod-template-hash=5d9447b574
Annotations:                         <none>
Status:                             Running
IP:                                  10.244.0.4
IPs:
  IP:                                10.244.0.4
Controlled By:                      ReplicaSet/myservice-5d9447b574
Containers:
  firstnameservice:
    Container ID:  docker://cda2849aa23cdc7e8793cfff36b2fa22efbd147ac82552b9416ddb5e5575a52
    Image:         5pierre/firstnameservice
    Image ID:      docker-pullable://5pierre/firstnameservice@sha256:d6abee5a92bcd09c4cd7f55609b9dba0270
    Port:         <none>
    Host Port:    <none>
    State:        Running
      Started:    Tue, 06 Jan 2026 20:04:12 +0100
    Ready:        True
    Restart Count: 0
    Environment:  <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-fjslr (ro)
```

On récupère ensuite l'adresse IP:

IP: 10.244.0.4

On note que cette adresse IP est éphémère puisque les pods peuvent être supprimés et remplacés par un nouveau.

On peut ensuite récupérer et visualiser le déploiement dans le tableau de bord de Minikube avec :

`minikube dashboard`

Ensuite pour explorer l'intérieur d'un conteneur de pod en mode interactif, on utilise :

`kubectl exec -it myservice-5d9447b574-4xcjv -- /bin/bash`

```

PS C:\Windows\system32> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
myservice-5d9447b574-4xcjv         1/1     Running   0           161m
PS C:\Windows\system32> kubectl exec -it myservice-5d9447b574-4xcjv -- /bin/bash
>>
root@myservice-5d9447b574-4xcjv:/var/www/html# ls
index.php
root@myservice-5d9447b574-4xcjv:/var/www/html# exit
exit
PS C:\Windows\system32>

```

3. Expose HTTP and HTTPS route using NodePort

Pour rendre l'application accessible depuis l'extérieur du cluster, nous avons exposé le déploiement via un service de type NodePort :

```
kubectl expose deployment myservice --type=NodePort --port=8080
```

Ensuite, nous avons récupéré l'URL pour accéder à l'application depuis un navigateur :

```
minikube service myservice --url
```

```

exit
PS C:\Windows\system32> kubectl expose deployment myservice --type=NodePort --port=8080
>>
service/myservice exposed
PS C:\Windows\system32> minikube service myservice --url
http://127.0.0.1:57655
! Comme vous utilisez un pilote Docker sur windows, le terminal doit être ouvert pour l'exécuter.

```

4. Scaling and load balancing

Afin de vérifier le bon fonctionnement du déploiement, nous avons commencé par contrôler l'état du déploiement myservice. Cette étape permet de s'assurer que le déploiement est bien actif et que le nombre de pods disponibles correspond au nombre attendu :

```
kubectl get deployments
```

Nous avons ensuite listé les services actifs afin d'identifier le service actuellement associé au déploiement :

```
kubectl get services
```

```
PS C:\Windows\system32> kubectl get deployments
NAME          READY    UP-TO-DATE    AVAILABLE    AGE
myservice     1/1      1             1             3h23m
PS C:\Windows\system32> kubectl get services
NAME          TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
kubernetes    ClusterIP     10.96.0.1     <none>         443/TCP          12h
myservice     NodePort      10.97.173.24  <none>         8080:32524/TCP   28m
PS C:\Windows\system32>
```

Ensuite il est nécessaire de supprimer le service existant pour éviter tout conflit. Le service a donc été supprimé avec la commande suivante :

```
kubectl delete service serviceName
```

Enfin, le déploiement a été exposé à nouveau afin de permettre la mise en place du load balancing :

```
kubectl expose deployment myservice --type=LoadBalancer --port=8080
```

```
minikube service myservice --url
```

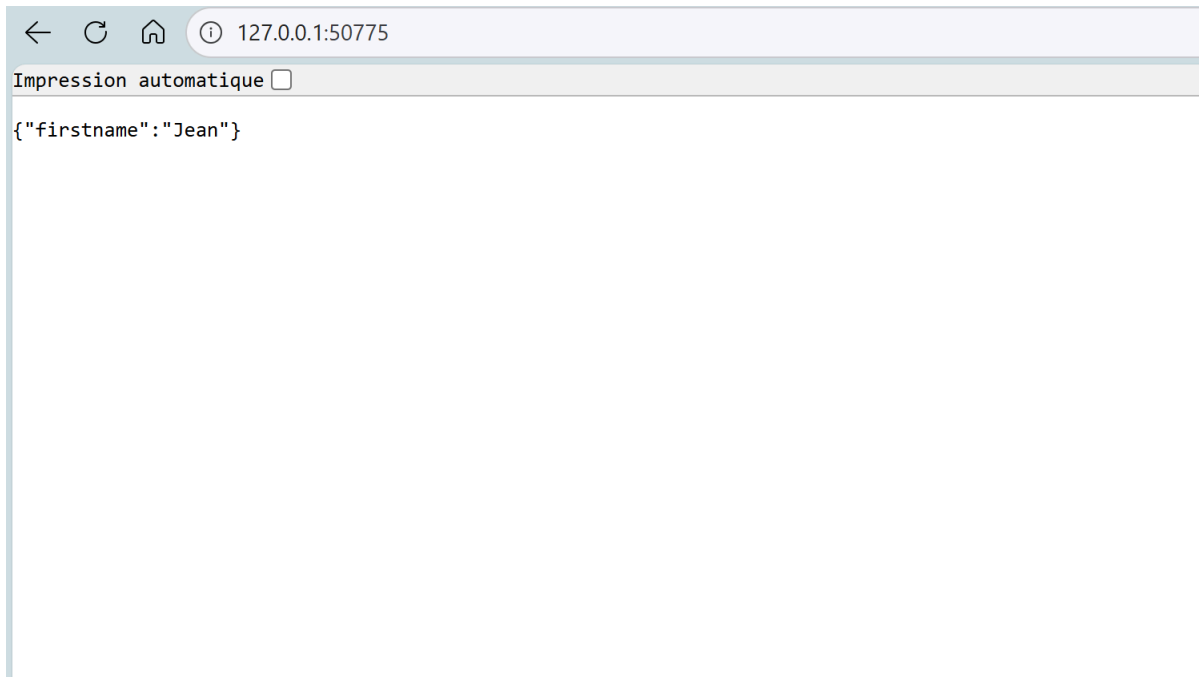
```
>> C:\Windows\system32>
NAME          TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
kubernetes    ClusterIP     10.96.0.1     <none>         443/TCP          12h
myservice     NodePort      10.97.173.24  <none>         8080:32524/TCP   28m
PS C:\Windows\system32> kubectl delete service myservice
service "myservice" deleted from default namespace
PS C:\Windows\system32> kubectl expose deployment myservice --type=LoadBalancer --port=8080
service/myservice exposed
PS C:\Windows\system32> minikube service myservice --url
http://127.0.0.1:49894
! Comme vous utilisez un pilote Docker sur windows, le terminal doit être ouvert pour l'exécuter.
```

j'ai adapté la commande car mon image Docker (5pierre/firstnameservice) utilise Apache, qui écoute par défaut sur le port 80. et la commande "kubectl expose" essaie d'utiliser le port 8080. donc pour éviter une erreur j'ai utilisé ceci:

```
kubectl expose deployment myservice --type=LoadBalancer --port=8080
```

```
PS C:\Windows\system32> kubectl expose deployment myservice --type=NodePort --port=8080 --target-port=80
service/myservice exposed
PS C:\Windows\system32> minikube service myservice --url
http://127.0.0.1:50775
! Comme vous utilisez un pilote Docker sur windows, le terminal doit être ouvert pour l'exécuter.
```

le code php de l'image est bien affiché



5. Rolling updates

Les Rolling Updates permettent de mettre à jour une application déployée sur Kubernetes sans interruption de service, en remplaçant progressivement les anciens pods par de nouveaux.

Mise à jour de l'image du déploiement :

```
kubectl set image deployment/myservice  
firstnameservice=5pierre/firstnameservice:v2
```

Vérification de l'état de la mise à jour :

```
kubectl rollout status deployment/myservice
```

Annulation de la mise à jour (Rollback) :

```
kubectl rollout undo deployment/myservice
```

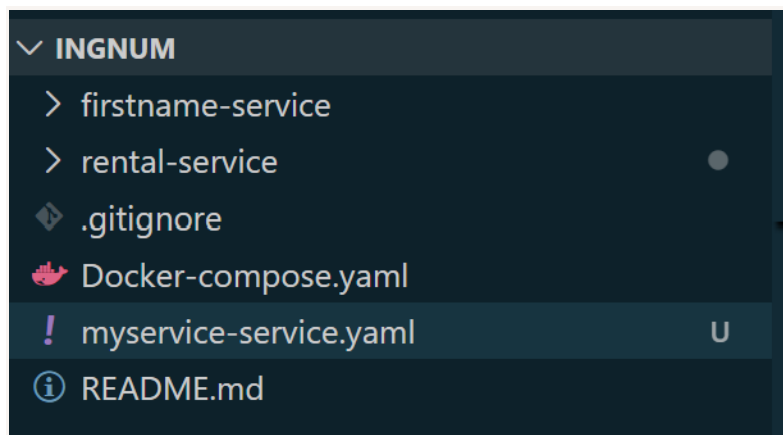
```
PS C:\Windows\system32> kubectl set image deployment/myservice firstnameservice=5pierre/firstnameservice  
deployment.apps/myservice image updated  
PS C:\Windows\system32> kubectl rollout status deployment/myservice  
Waiting for deployment "myservice" rollout to finish: 1 old replicas are pending termination...  
PS C:\Windows\system32> kubectl rollout undo deployment/myservice  
deployment.apps/myservice rolled back  
PS C:\Windows\system32>
```

6. Create a deployment and a service using a yaml file

Appliquer le déploiement et le service NodePort :

```
kubectl apply -f myservice-deployment.yaml
```

```
PS C:\Users\pllie\OneDrive\Documents\Efrei_2023-2025\Cours-L3\docker\exo docker tp1\ingnum> kubectl apply -f myservice-service.yaml
deployment.apps/first-service unchanged
service/first-service unchanged
deployment.apps/rental-service unchanged
service/rental-service created
PS C:\Users\pllie\OneDrive\Documents\Efrei_2023-2025\Cours-L3\docker\exo docker tp1\ingnum> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
first-service-759f7b6857-rhvvv      1/1     Running   0           2m32s
myservice-5d9447b574-4xcjv          1/1     Running   0           15h
rental-service-f644f4b6b-fjc78      1/1     Running   0           2m32s
PS C:\Users\pllie\OneDrive\Documents\Efrei_2023-2025\Cours-L3\docker\exo docker tp1\ingnum> |
```



7. Communication entre microservices

Pour permettre la communication, j'ai modifié l'URL dans le code Java (application.properties) pour utiliser le nom DNS interne de Kubernetes :

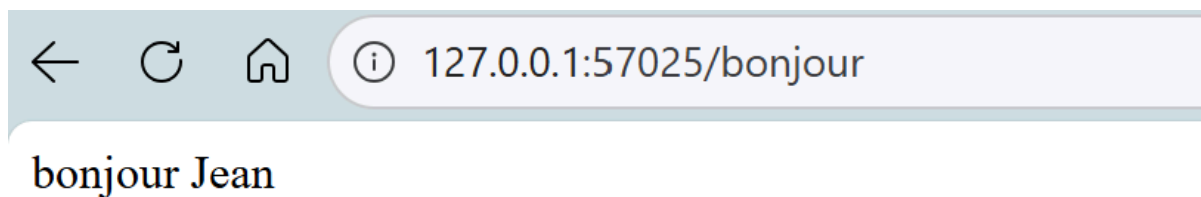
```
http://firstname-service.default.svc.cluster.local:80/index.php
```

Pour déployer proprement, j'ai utilisé un nouveau fichier `final-architecture.yaml`

J'applique ensuite la configuration :

```
kubectl apply -f final-architecture.yaml
```

```
PS C:\Users\pllie\OneDrive\Documents\Efrei_2023-2025\Cours-L3\docker\exo docker tp1\ingnum> kubectl apply -f final-architect
re.yaml
deployment.apps/firstname-deployment unchanged
service/firstname-service unchanged
deployment.apps/rental-deployment unchanged
service/rental-service unchanged
PS C:\Users\pllie\OneDrive\Documents\Efrei_2023-2025\Cours-L3\docker\exo docker tp1\ingnum> minikube service rental-service
-url
http://127.0.0.1:57025
! Because you are using a Docker driver on windows, the terminal needs to be open to run it.
```



8. Gateway (serveur porte d'entrée devant nos microservices)

L'objectif est d'utiliser un point d'entrée unique (Ingress) pour router le trafic vers nos différents microservices, plutôt que d'exposer chaque service individuellement.

Activation du contrôleur Ingress NGINX et vérification de son fonctionnement:

```
minikube addons enable ingress
```

```
kubectl get pods -n ingress-nginx
```

```

PS C:\Windows\system32> minikube addons enable ingress
* ingress est un add-on maintenu par Kubernetes. Pour toute question, contactez minikube sur GitHub.
Vous pouvez consulter la liste des mainteneurs de minikube sur : https://github.com/kubernetes/minikube/blob/master/OWNERS
* Après que le module est activé, veuillez exécuter "minikube tunnel" et vos ressources ingress seront disponibles à "127.0.0.1"
  - Utilisation de l'image registry.k8s.io/ingress-nginx/controller:v1.13.2
  - Utilisation de l'image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.2
  - Utilisation de l'image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.2
* Vérification du module ingress...
* Le module 'ingress' est activé
PS C:\Windows\system32> kubectl get pods -n ingress-nginx
NAME                                READY   STATUS    RESTARTS   AGE
ingress-nginx-admission-create-97gbb 0/1     Completed 0           96s
ingress-nginx-admission-patch-v7hk9   0/1     Completed 0           96s
ingress-nginx-controller-9cc49f96f-k69xx 1/1     Running   0           96s
PS C:\Windows\system32>

```

Déploiement de la configuration Ingress et vérification de son fonctionnement : à l'aide du fichier ingress.yml (sur github),

```
kubectl apply -f ingress.yml
```

Et récupération de l'adresse IP de l'Ingress :

```
kubectl get ingress
```

```

PS C:\Users\pllie\OneDrive\Documents\Efrei_2023-2025\Cours-L3\docker\exo docker tp1\ingnum> kubectl apply -f ingress.yml
ingress.networking.k8s.io/example-ingress created
PS C:\Users\pllie\OneDrive\Documents\Efrei_2023-2025\Cours-L3\docker\exo docker tp1\ingnum> kubectl get ingress
NAME          CLASS  HOSTS          ADDRESS      PORTS      AGE
example-ingress  nginx  myservice.info  127.0.0.1    80         15s
PS C:\Users\pllie\OneDrive\Documents\Efrei_2023-2025\Cours-L3\docker\exo docker tp1\ingnum>

```

ensuite il faut configurer le DNS (Windows hosts) en y ajoutant :

```
127.0.0.1 myservice.info
```

Puis rendre l'adresse accessible sur Windows, avec l'activation du tunnel:

```
minikube addons enable ingress-dns
```

```
minikube tunnel
```

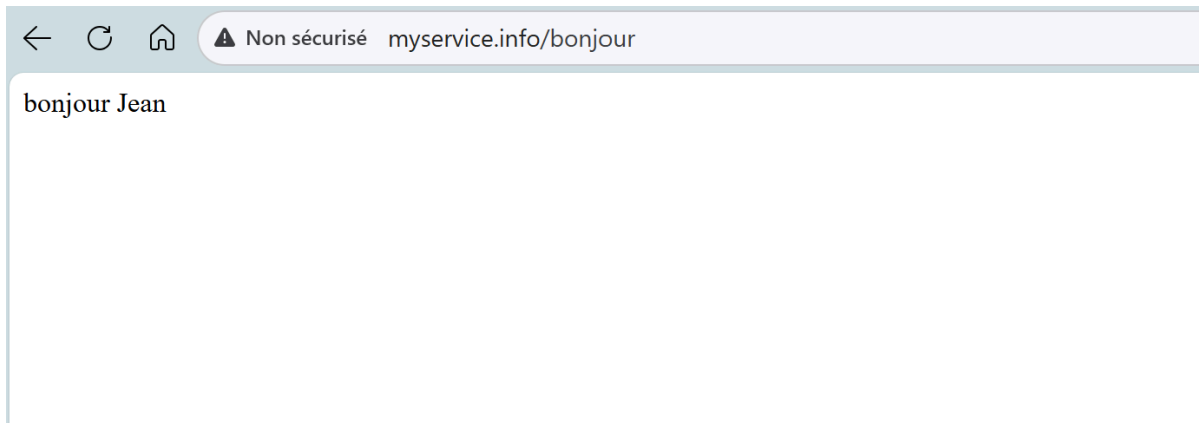
```

PS C:\Windows\system32> minikube addons enable ingress-dns
* ingress-dns est un add-on maintenu par minikube. Pour toute question, contactez minikube sur GitHub.
Vous pouvez consulter la liste des mainteneurs de minikube sur : https://github.com/kubernetes/minikube/blob/master/OWNERS
* Après que le module est activé, veuillez exécuter "minikube tunnel" et vos ressources ingress seront disponibles à "127.0.0.1"
  - Utilisation de l'image docker.io/kicbase/minikube-ingress-dns:0.0.4
* Le module 'ingress-dns' est activé
PS C:\Windows\system32> minikube tunnel
* Tunnel démarré avec succès

* REMARQUE : veuillez ne pas fermer ce terminal car ce processus doit rester actif pour que le tunnel soit accessible...

* Tunnel de démarrage pour le service rental-service.
! Accéder aux ports inférieurs à 1024 peut échouer sur Windows avec les clients OpenSSH antérieurs à v8.1. Pour plus d'info voir: https://minikube.sigs.k8s.io/docs/handbook/accessing/#access-to-ports-1024-on-windows-requires-root-permission
* Tunnel de démarrage pour le service example-ingress.\

```

ensuite j'ai créé un deuxième déploiement et son service, puis en ajoutant une nouvelle route au fichier ingress.yml(visible sur gitbub.

